

15. Bölüm

LINQ Sorguları

15.1 LINQ Nedir?

Dile Entegre Sorgu LINQ(Language INtegrated Query) anlamına gelir. Çeşitli veri kaynakları ve biçimleri arasında verilerle çalışmak için tutarlı bir model sunarak bir sorgu dilini programlama diline entegre eden bir kavramdır; XML belgelerinde, SQL veri tabanlarında, ADO.NET Veri Kümelerinde, .NET koleksiyonlarında ve LINQ sağlayıcısının sunmuş olduğu biçimlerde verileri sorgulamak ve dönüştürmek için aynı temel kodlama kalıplarını kullanırsınız.

LINQ, çeşitli veri kaynaklarından gelen verileri tek tip bir şekilde sorgulamamıza olanak tanıyan ortak bir söz dizimi sağlar. Bu, tek bir LINQ sorgusu kullanarak SQL Server veri tabanı, XML belgeleri, ADO.NET Veri Kümeleri ve Koleksiyonlar, Jenerik gibi çeşitli veri kaynaklarından veri alabileceğimiz anlamına gelir.

```
List<İl> iller = new List<İl> {  
    new İl { ilTrafikKodu = 1, Adı = "Adana" },  
    new İl { ilTrafikKodu = 34, Adı = "İstanbul" },  
    new İl { ilTrafikKodu = 5, Adı = "Amasya" },  
    new İl { ilTrafikKodu = 6, Adı = "Ankara" },  
    new İl { ilTrafikKodu = 7, Adı = "Antalya" },  
    new İl { ilTrafikKodu = 16, Adı = "Bursa" },  
    new İl { ilTrafikKodu = 10, Adı = "Balıkesir" },  
    new İl { ilTrafikKodu = 61, Adı = "Trabzon" }  
};
```

// Örneklerdeki Sorgular Buraya Gelecek

```
struct İl  
{  
    public int ilTrafikKodu;  
    public string Adı;  
}
```

Yukarıdaki gibi bir veri kaynağını göz önüne aldığımızda LINQ sorguları iki şekilde yazılabilir. Başlıklar halinde verilmiştir.

§15.1.1 Sorgu Şeklinde

SQL'e benzer ve genellikle SQL'e aşina olanlar için daha okunabilir. Bir **from** ifadesiyle başlar, ardından bir aralık değişkeni gelir ve **where**, **select**, **group**, **join** vb. gibi standart sorgu işlemlerini içerir.

```
from object in datasource  
where condition  
select object
```

Yukarıdaki söz dizimine sahip sorguya örnek aşağıdaki kod verilebilir;

```
var query = from i in iller  
            where i.Adı == "Ankara"  
            select i.Adı;
```

§15.1.2 Akıcı (fluent) Şekilde

Genişleme yöntemleri (**extension method**) ve lambda ifadeleri kullanır. Daha öz olabilir ve karmaşık sorgular yazarken tercih edilir çünkü okunması ve oluşturulması daha kolay olabilir.

```
datasource.ConditionMethod().SelectionMethod()
```

Yukarıdaki söz dizimine sahip sorguya örnek aşağıdaki kod verilebilir;

```
var query = iller.Where(i => i.Adı == "Ankara").Select(i => i.Adı);  
IEnumerable ve IQueryable, veri işleme ve sorgulama için sıklıkla kullanılan iki ara yüzdür (interface),
```

ancak farklı amaçlara hizmet ederler ve farklı davranışlara sahiptirler. Aşağıdaki örnekte `query` değişkenleri `IEnumerable<string>?` veri tipine sahiptir.

```
IEnumerable<int> queryIlKodları = iller.Select(i => i.ilTrafikKodu);
IEnumerable<string> queryIlListesi = iller.Where(i => i.Adı.Contains("an")).Select(i => i.Adı);
```

LINQ

LINQ sorgularının tanımlandığı anda değil, üzerinde `foreach` ile dönlüldüğünde veya `.ToList()` gibi yöntemler çağrıldığında çalıştırıldığına veri döndürür. Bu, **LINQ'in en temel mantığıdır!**

15.2 IEnumerable ve IEnumerator Arayüzleri

LINQ fonksiyonları hem bir `IEnumerable<TSource>` üzerinde çalışır ve bir `IEnumerator<TResult>` döndürür. Bu arayüzler daha önce 14.2.5 başlığında anlatılmıştır. Bu arayüzler .NET Framework'ün önemli bir parçasıdır ve bir nesne koleksiyonu üzerinde yineleme (**iteration**) yapmak için kullanılır. Yukarıdaki örnekte yapılan sorgulamalar aşağıdaki şekilde yinelenabilir;

```
foreach (var item in queryIlKodları)
    Console.WriteLine($"{item} ");
// 1 34 5 6 7 16 10 61
foreach (var item in queryIlListesi)
    Console.WriteLine($"{item} ");
// Adana İstanbul
```

15.3 IQueryable Arayüzü

`IQueryable`, bir veri kaynağından veri sorgulamak için kullanılan bir ara yüzdür. `System.Linq` isim uzayının bir parçasıdır ve önemli bir bileşendir. Bellek içi koleksiyonlar üzerinde yineleme yapmak için kullanılan `IEnumerable` ara yüzünün aksine, `IQueryable`, nesne numaralandırılana kadar sorgunun yürütülmediği veri kaynaklarını sorgulamak için tasarlanmıştır. Bu, özellikle veri tabanları gibi uzak veri kaynakları için yararlıdır ve sorgunun sonucu tarafında yürütülmesine izin vererek verimli sorgulamayı mümkün kılar.

```
IQueryable<İl> bazıİller = iller.AsQueryable().Where(i => i.ilTrafikKodu <30);
foreach (var item in bazıİller) //Sorgu sonucu İl nesneleri döner
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*Program Çıktısı:
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
*/
```

`IEnumerable` istemci belleğinde yürütülürken, `IQueryable` veri kaynağında yürütülür. `IEnumerable` nesneler ve bellek içi verilerle çalışmak için uygundur. `IQueryable`, SQL veri tabanları ile etkileşim kurması için uygundur. `IQueryable`, veri tabanının verileri optimize etmesine ve filtrelemesine izin verdiği için büyük veri kümeleri için daha iyi performans gösterebilir.

15.4 LINQ Genişleme Yöntemleri

Genişleme yöntemleri (**extension method**), yeni bir türetilmiş tip oluşturmadan, yeniden derlemeden veya orijinal türü başka şekilde değiştirmeden mevcut türlere davranışı genişletir ve ekler. Genişleme yöntemleri konusu 8.18 başlığında incelenmişti.

§15.4.1 Where

```
var filteredResult = iller.Where(item => item.Adı == "Ankara");
foreach (var item in filteredResult)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 6, Adı: Ankara
*/
```

§15.4.2 Select

```
var projectedResult = iller.Select(item => new { ilTrafikKodu = 61, Adı = "Trabzon" });
foreach (var item in projectedResult)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/* iller koleksiyonunda ne kadar eleman varsa o kadar yeni il ekler
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 61, Adı: Trabzon
*/
foreach (var item in iller)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 61, Adı: Trabzon
*/
```

§15.4.3 OrderBy ve OrderByDescending

```
var sortedResult = iller.OrderBy(item => item.ilTrafikKodu);
foreach (var item in sortedResult)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 61, Adı: Trabzon
*/
var sortedDescendingResult = iller.OrderByDescending(item => item.Adı);
foreach (var item in sortedDescendingResult)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 1, Adı: Adana
*/
```

§15.4.4 Group By

```
List<İl> iller = new List<İl> {
    new İl { ilTrafikKodu = 1, Adı = "Adana", Bölge="Akdeniz" },
    new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 5, Adı = "Amasya", Bölge = "Karadeniz" },
    new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
```

```

new İl { ilTrafikKodu = 7, Adı = "Antalya", Bölge = "Akdeniz"},
new İl { ilTrafikKodu = 16, Adı = "Bursa", Bölge = "Marmara"},
new İl { ilTrafikKodu = 10, Adı = "Balıkesir", Bölge = "Marmara" },
new İl { ilTrafikKodu = 61, Adı = "Trabzon", Bölge = "Karadeniz" }
};
IEnumerable<IGrouping<string, İl>> groupedResult = iller.GroupBy(item => item.Bölge);
foreach (var item in groupedResult)
{
    Console.WriteLine($"{item.Key} "); //Akdeniz Marmara Karadeniz İç Anadolu
}
struct İl
{
    public int ilTrafikKodu;
    public string Adı;
    public string Bölge;
}

```

§15.4.5 First ve Last

Bir dizinin ilk öğesini veya sağlanan öngörüye uyan ilk veya son elemanı döndürür. Dizi hiçbir öğe içermiyorsa, şu iletiyle bir `InvalidOperationException` istisnası oluşur.

```

new[] { "a" }.First(); // "a"
new[] { "a", "b" }.First(); // "a"
new[] { 1, 2 }.First(); // 1
new[] { "a", "b" }.First(x => x.Equals("b")); // "b"
new[] { 1, 2 }.First(x => x > 1); // 2
new[] { "ba", "be" }.First(x => x.Contains("b")); // "ba"
//new[] { "ca", "ce" }.First(x => x.Contains("b")); // Hata: InvalidOperationException
new[] { "a" }.Last(); // "a"
new[] { "a", "b" }.Last(); // "b"
Console.WriteLine(new[] { 1, 2 }.Last()); // 2
new[] { "a", "b" }.Last(x => x.Equals("a")); // "a"
Console.WriteLine(new[] { 1, 2 }.Last(x => x < 2)); // 1
new[] { "ba", "be" }.Last(x => x.Contains("b")); // "be"
//new[] { "ca", "ce" }.Last(x => x.Contains("b")); // Hata: InvalidOperationException

```

§15.4.6 FirstOrDefault ve LastOrDefault

Bir dizinin ilk öğesini veya sağlanan süzgece uyan ilk elemanı veya son elemanı döndürür. Dizi hiçbir öğe içermiyorsa veya sağlanan fonksiyona uyan hiçbir öğe yoksa, `default(T)` kullanılarak dizi veri tipinin varsayılan değeri döndürülür.

```

new[] { "a" }.FirstOrDefault(); // "a"
new[] { "a", "b" }.FirstOrDefault(); // "a"
new[] { 1, 2 }.FirstOrDefault(); // 1
new[] { "a", "b" }.FirstOrDefault(x => x.Equals("b")); // "b"
new[] { 1, 2 }.FirstOrDefault(x => x > 1); // 2
new[] { "ba", "be" }.FirstOrDefault(x => x.Contains("b")); // "ba"
new[] { 1, 2 }.FirstOrDefault(x => x > 2); // 0
new[] { "ca", "ce" }.FirstOrDefault(x => x.Contains("b")); // null
new[] { "a" }.LastOrDefault(); // "a"
new[] { "a", "b" }.LastOrDefault(); // "b"
new[] { 1, 2 }.LastOrDefault(); // 2
new[] { "a", "b" }.LastOrDefault(x => x.Equals("a")); // "a"
new[] { 1, 2 }.LastOrDefault(x => x < 2); // 1
new[] { "ba", "be" }.LastOrDefault(x => x.Contains("b")); // "b3"
new[] { 1, 2 }.LastOrDefault(x => x > 2); // 0
new[] { "ca", "ce" }.LastOrDefault(x => x.Contains("b")); // null

```

§15.4.7 Single

Bir dizinin tek elemanı varsa veya sağlanan süzgece uyan tek elemanı döndürür. Dizi hiçbir öğe içermiyorsa, şu iletiyle bir `InvalidOperationException` istisnası oluşur.

```

new[] { "a" }.Single(); // "a"
//new[] { "a", "b" }.Single(); // InvalidOperationException

```

```
new[] { 1 }.Single(); //1
new[] { "a", "b" }.Single(x => x.Equals("b")); // "b"
new[] { 1, 2 }.Single(x => x < 2); //1
//new[] { "a", "b" }.Single(x => x.Equals("c")); // InvalidOperationException
//new[] { 1, 2 }.Single(x => x > 2); //InvalidOperationException
//new string[0].Single(); // InvalidOperationException
```

§15.4.8 SingleOrDefault

Bir dizinin son öğesini veya sağlanan süzgece uyan son öğeyi döndürür. Dizi hiçbir öğe içermiyorsa veya sağlanan fonksiyona uyan hiçbir öğe yoksa, `default(T)` kullanılarak dizi veri tipinin varsayılan değeri döndürülür.

```
new[] { "a" }.SingleOrDefault(); // "a"
//new[] { "a", "b" }.SingleOrDefault(); // InvalidOperationException
new[] { 1 }.SingleOrDefault(); //1
new[] { "a", "b" }.SingleOrDefault(x => x.Equals("b")); // "b"
new[] { 1, 2 }.SingleOrDefault(x => x > 1); //2
new[] { "a", "b" }.SingleOrDefault(x => x.Equals("c")); // null
new[] { 1, 2 }.SingleOrDefault(x => x > 2); // 0
```

§15.4.9 Sum, Min, Max ve Average

```
var total = iller.Sum(item => item.ilTrafikKodu);
Console.WriteLine($"Toplam İl Trafik Kodu: {total}");
var average = iller.Average(item => item.ilTrafikKodu);
Console.WriteLine($"Ortalama İl Trafik Kodu: {average}");
var min = iller.Min(item => item.ilTrafikKodu);
Console.WriteLine($"En Küçük İl Trafik Kodu: {min}");
var max = iller.Max(item => item.ilTrafikKodu);
Console.WriteLine($"En Büyük İl Trafik Kodu: {max}");
```

§15.4.10 Any ve All

Herhangi bir eleman veya tüm dizi elemanlarını bir koşulu karşılayıp karşılamadığını kontrol eder.

```
bool hasItems = iller.Any();
Console.WriteLine(hasItems); // true
bool allMatchCondition = iller.All(item => item.ilTrafikKodu < 70);
Console.WriteLine(allMatchCondition); // true
bool anotherMatchCondition = iller.All(item => item.ilTrafikKodu > 70);
Console.WriteLine(anotherMatchCondition); // false
```

§15.4.11 Count

Bir dizideki öğeleri sayar, isteğe bağlı olarak koşulla verilebilir.

```
int count = iller.Count();
Console.WriteLine($"Toplam İl Sayısı: {count}"); // 8
int conditionalCount = iller.Count(item => item.Bölge == "Karadeniz");
Console.WriteLine($"Karadeniz Bölgesindeki İl Sayısı: {conditionalCount}"); // 2
```

§15.4.12 FindAll

Verilen ifadeye uygun bir dizideki öğeleri döndürür.

```
List<int> girdiler = new List<int>() { 10, 20, 30, 40, 50, 60 };
Console.WriteLine("Girdiler: ");
foreach (var değer in girdiler)
    Console.WriteLine("{0} ", değer);
// Girdiler: 10 20 30 40 50 60
Console.WriteLine();
List<int> üçeBölünenler = girdiler.FindAll(x => (x % 3) == 0);
Console.WriteLine("Üçe Bölünen Girdiler: ");
foreach (var değer in üçeBölünenler)
    Console.WriteLine("{0} ", değer);
// Üçe Bölünen Girdiler: 30 60
```

§15.4.13 Distinct

Bir diziden tekil olan veya diğerlerinden farklı elemanları döndürür.

```
List<İl> iller = new List<İl> {
    new İl { ilTrafikKodu = 1, Adı = "Adana" , Bölge="Akdeniz" },
    new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
    new İl { ilTrafikKodu = 1, Adı = "Adana" , Bölge="Akdeniz" },
    new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
    new İl { ilTrafikKodu = 1, Adı = "Adana" , Bölge="Akdeniz" },
    new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
};
Console.WriteLine("Listenin Tekil Olan Elemanları:");
var tekilliste = iller.Distinct();
foreach (var item in tekilliste)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
Listenin Tekil Olan Elemanları:
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 6, Adı: Ankara
*/
struct İl
{
    public int ilTrafikKodu;
    public string Adı;
    public string Bölge;
}
```

§15.4.14 Intersect, Except ve Union

Diziler üzerinde kesişim, fark ve birleşim gibi küme işlemlerini gerçekleştirir.

```
List<İl> iller1 = new List<İl> {
    new İl { ilTrafikKodu = 1, Adı = "Adana" , Bölge="Akdeniz" },
    new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 5, Adı = "Amasya", Bölge = "Karadeniz" },
    new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
    new İl { ilTrafikKodu = 7, Adı = "Antalya", Bölge = "Akdeniz"},
    new İl { ilTrafikKodu = 16, Adı = "Bursa", Bölge = "Marmara"},
    new İl { ilTrafikKodu = 10, Adı = "Balıkesir", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 61, Adı = "Trabzon", Bölge = "Karadeniz" }
};
List<İl> iller2 = new List<İl> {
    new İl { ilTrafikKodu = 34, Adı = "İstanbul", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 6, Adı = "Ankara", Bölge = "İç Anadolu" },
    new İl { ilTrafikKodu = 7, Adı = "Antalya", Bölge = "Akdeniz"},
    new İl { ilTrafikKodu = 16, Adı = "Bursa", Bölge = "Marmara"},
    new İl { ilTrafikKodu = 10, Adı = "Balıkesir", Bölge = "Marmara" },
    new İl { ilTrafikKodu = 61, Adı = "Trabzon", Bölge = "Karadeniz" },
    new İl { ilTrafikKodu = 35, Adı = "İzmir", Bölge = "Ege" },
    new İl { ilTrafikKodu = 45, Adı = "Manisa", Bölge = "Ege" },
    new İl { ilTrafikKodu = 67, Adı = "Zonguldak", Bölge = "Karadeniz" }
};
Console.WriteLine("İki Listenin Kesişim Kümesi:");
var kesişimKümesi = iller1.Intersect(iller2);
foreach (var item in kesişimKümesi)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
*/
```

```

İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 61, Adı: Trabzon
*/
Console.WriteLine("İki Listenin Birleşim Kümesi:");
var birleşimKümesi = iller1.Union(iller2);
foreach (var item in birleşimKümesi)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 34, Adı: İstanbul
İl Trafik Kodu: 5, Adı: Amasya
İl Trafik Kodu: 6, Adı: Ankara
İl Trafik Kodu: 7, Adı: Antalya
İl Trafik Kodu: 16, Adı: Bursa
İl Trafik Kodu: 10, Adı: Balıkesir
İl Trafik Kodu: 61, Adı: Trabzon
İl Trafik Kodu: 35, Adı: İzmir
İl Trafik Kodu: 45, Adı: Manisa
İl Trafik Kodu: 67, Adı: Zonguldak
*/
Console.WriteLine("İki Listenin Fark Kümesi:");
var farkKümesi = iller1.Except(iller2);
foreach (var item in farkKümesi)
    Console.WriteLine($"İl Trafik Kodu: {item.ilTrafikKodu}, Adı: {item.Adı}");
/*
İl Trafik Kodu: 1, Adı: Adana
İl Trafik Kodu: 5, Adı: Amasya
*/
struct il
{
    public int ilTrafikKodu;
    public string Adı;
    public string Bölge;
}

```

§15.4.15 OfType

Elamanları değişik veri tiplerinde olan koleksiyonlarda filtreleme yapar.

```

System.Collections.ArrayList meyveler = new()
{
    "Mango",
    "Portakal",
    null,
    "İğde",
    3.0,
    "Armut",
    10,
    "Çağla"
};
IEnumerable<string> query1 = meyveler.OfType<string>();
Console.WriteLine("'string' veri tipli elemanlar:");
foreach (string meyve in query1)
    Console.WriteLine(meyve);
/*
'string' veri tipli elemanlar:
Mango
Portakal
İğde
Armut
Çağla
*/

```



```

IEnumerable<string> query2 = meyveler.OfType<string>().Where(fruit => fruit.Contains('ğ',
    ↳ StringComparison.CurrentCultureIgnoreCase));
Console.WriteLine("\n'ğ' harfli meyveler:");
foreach (string meyve in query2)
    Console.WriteLine(meyve);
/*
'ğ' harfli meyveler:
İğde
Çağla
*/

```

§15.4.16 SelectMany

Bu yöntem bir `IEnumerable<IEnumerable<T>>` ögesini bir `IEnumerable<T>` ögesi olarak değiştirir. Kaynak `IEnumerable`'da bulunan `IEnumerable` örnekleri içindeki tüm `T` öğeleri tek bir `IEnumerable`'da birleştirilir.

```

var kelimeler = new[] { "ali,ahmet,cemal", "damla,efe", "fatma" };
var virgilleayrilmiskelimeler = kelimeler.SelectMany(x => x.Split(','));
foreach (var item in virgilleayrilmiskelimeler)
    Console.Write($"{item} ");
//ali ahmet cemal damla efe fatma
Console.WriteLine();
Yemek[] yemekler = new[] {
    new Yemek() {
        Meyveler = new [] { new Meyve { Adı="Elma"}, new Meyve { Adı = "Armut" } },
    new Yemek() {
        Meyveler = new [] { new Meyve { Adı = "Çilek"}, new Meyve { Adı = "Ahududu" } }
    }
};
var tümYemekler = yemekler.SelectMany(y => y.Meyveler.Select(m => m.Adı));
foreach (var item in tümYemekler)
    Console.Write($"{item} ");
//Elma Armut Çilek Ahududu
class Meyve
{
    public required string Adı { get; set; }
}
class Yemek
{
    public required Meyve[] Meyveler { get; set; }
}

```

§15.4.17 Join

Birleştirmeler (*join*), ortak bir anahtar aracılığıyla veri tutan farklı listeleri veya tabloları birleştirmek için kullanılır. SQL'de olan türde birleştirmeler desteklenir: İç (*inner*), Sol (*left*), Sağ (*right*), Çapraz (*cross*) ve Tam Dış (*full outer*) birleştirmeler. Aşağıdaki iki tabloyu göz önüne alalım.

```

var sol = new List<string>() { "armut", "muz", "elma" }; // Sol (Left) tablo
var sağ = new List<string>() { "armut", "mango", "elma", "çilek" }; // Sağ (Right)

```

İç Birleştirme (Inner Join):

```

var innerjoin = from l in sol
                join s in sağ on l equals s
                select new { İLK = l, İKİNCİ = s };
var innerjoinfluent = sol.Join(sağ, l => l, s => s, (l, s) => new { İLK = l, İKİNCİ = s });
foreach (var item in innerjoinfluent)
    Console.Write($"{item.İLK}-{item.İKİNCİ} ");
//armut-armut elma-elma

```

Sol Bileştirme (Left Outer Join):

```

var leftOuterJoin = from l in sol
                    join s in sağ on l equals s into tmp
                    from t in tmp.DefaultIfEmpty()

```



```

        select new { İLK = l, İKİNCİ = t };
var leftOuterJoin2 = from l in sol
                    from s in sağ.Where(x => x == l).DefaultIfEmpty()
                    select new { İLK = l, İKİNCİ = s };
var leftOuterJoinFluent = sol.GroupJoin(sağ, l => l, s => s, (l, s) => new { İLK = l, İKİNCİ= s
    ↳ }).SelectMany(temp => temp.İKİNCİ.DefaultIfEmpty(),(l, s) => new { İLK = l.İLK, İKİNCİ = s });
foreach (var item in leftOuterJoinFluent)
    Console.WriteLine($"{item.İLK}-{item.İKİNCİ} ");
//armut-armut muz-(null) elma-elma

```

Sağ Bileştirme (Right Outer Join):

```

var rightOuterJoin = from s in sağ
                    join l in sol on s equals l into temp
                    from t in temp.DefaultIfEmpty()
                    select new { İLK = t, İKİNCİ = s };
foreach (var item in rightOuterJoin)
    Console.WriteLine($"{item.İLK}-{item.İKİNCİ} ");
//armut-armut (null)-mango elma-elma (null)-çilek

```

Çapraz Birleştirme (Cross Join):

```

var CrossJoin = from f in sol
                from s in sağ
                select new { İLK=f, İKİNCİ=s };
foreach (var item in CrossJoin)
    Console.WriteLine($"{item.İLK}-{item.İKİNCİ}");
/*
armut-armut
armut-mango
armut-elma
armut-çilek
muz-armut
muz-mango
muz-elma
muz-çilek
elma-armut
elma-mango
elma-elma
elma-çilek
*/

```

Tam Dış Birleşme (Full Outer Join):

```

var leftOuterJoin = from l in sol
                    join s in sağ on l equals s into tmp
                    from t in tmp.DefaultIfEmpty()
                    select new { İLK = l, İKİNCİ = t };
var rightOuterJoin = from s in sağ
                    join l in sol on s equals l into temp
                    from t in temp.DefaultIfEmpty()
                    select new { İLK = t, İKİNCİ = s };
var fullOuterjoin = leftOuterJoin.Union(rightOuterJoin);
foreach (var item in fullOuterjoin)
    Console.WriteLine($"{item.İLK}-{item.İKİNCİ}");
/*
armut-armut
muz-(null)
elma-elma
(null)-mango
(null)-çilek
*/

```

§15.4.18 Skip ve Take

Skip yöntemi, kaynak koleksiyonun başlangıcından itibaren belirli sayıda elemanı hariç tutan bir koleksiyon döndürür. Hariç tutulan eleman sayısı, argüman olarak verilen sayıdır. Koleksiyonda argümanda belirtilenden daha az öge varsa, boş bir koleksiyon döndürülür.

Take yöntemi, kaynak koleksiyonun başlangıcından itibaren bir dizi eleman içeren bir koleksiyon döndürür. Dahil edilen eleman sayısı, argüman olarak verilen sayıdır. Koleksiyonda argümanda belirtilenden daha az eleman varsa, döndürülen koleksiyon kaynak koleksiyonuyla aynı elemanları içerecektir.

```
var values = new[] { 5, 4, 3, 2, 1 };
var skipTwo = values.Skip(2); // { 3, 2, 1 }
var takeThree = values.Take(3); // { 5, 4, 3 }
var skipOneTakeTwo = values.Skip(1).Take(2); // { 4, 3 }
var takeZero = values.Take(0); // IEnumerable<int> fakat elemanı yok
```

§15.4.19 Range ve Repeat

Enumerable'daki **Range** ve **Repeat** statik yöntemleri basit diziler oluşturmak için kullanılabilir.

```
var range = Enumerable.Range(1,100); // [1, 2, 3, ..., 98, 99, 100]
/* elemanları 1 den başlayıp 100'e kadar devam eden bir tamsayı dizisi oluşturur. */
var repeatedValues = Enumerable.Repeat("a", 3); // ["a","a","a"]
/* aynı elemandan birden fazla içeren dizi oluşturmak için kullanılır*/
```

§15.4.20 Contains

Belirtilen bir **IEqualityComparer<T>** kullanarak bir dizinin belirtilen bir elemanı içerip içermediğini belirler.

```
List<int> sayılar = new List<int> { 1, 2, 3, 4, 5 };
var sonuç1 = sayılar.Contains(4); // true
var sonuç2 = sayılar.Contains(8); // false
List<int> başkaSayılar = new List<int> { 4, 5, 6, 7 };
var sonuç3 = başkaSayılar.Where(item => sayılar.Contains(item)); //true (4,5)
```

§15.4.21 Zip

Zip yöntemi iki koleksiyon üzerinde etki eder. İki serideki her bir elemanı indise (konuma) göre eşleştirir. İki koleksiyondan gelen elemanları çiftler halinde işlemek için kullanırız. Seriler boyut olarak farklıysa, daha büyük serinin ekstra elemanları yok sayılır.

```
int[] sayılar = { 3, 4, 5, 7 };
string[] sözcükler = { "üç", "dört", "beş", "yedi", "göz ardı edilecek" };
IEnumerable<string> zip = sayılar.Zip(sözcükler, (n, s) => n + "=" + s);
foreach (var item in zip)
    Console.WriteLine(item);
/*
3=üç
4=dört
5=beş
7=yedi
*/
```

§15.4.22 Aggregate

Bir dizi üzerinde bir biriktirici fonksiyonu uygular.

```
int[] sayıListesi = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
int toplam = sayıListesi.Aggregate((aratoplama, şimdiki) => aratoplama + şimdiki);
Console.WriteLine(toplam); // 55
string[] stringList = { "Merhaba", "C#", "!" };
string birleşikmetin = stringList.Aggregate((prev, current) => prev + " " + current);
Console.WriteLine(birleşikmetin); // Merhaba C# !
```

15.5 Let Anahtar Kelimesi

Bir LINQ ifadesinin içinde bir değişken tanımlamak için **let** anahtar sözcüğünü kullanabilirsiniz. Bu genellikle ara alt sorguların sonuçlarını depolamak için yapılır.

```
int[] sayılar = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
```

```

var ortalamaÜstükareler = from sayı in sayılar
                           let ortalama = sayılar.Average()
                           let kare = Math.Pow(sayı, 2)
                           where kare > ortalama
                           select sayı;
Console.WriteLine("Ortalama: {0}", sayılar.Average());
foreach (int n in ortalamaÜstükareler)
    Console.WriteLine("{0} sayısının karesi {1} Ortalamayı Geçti!", n, Math.Pow(n,2));
/*
Ortalama: 4,5
3 sayısının karesi 9 Ortalamayı Geçti!
4 sayısının karesi 16 Ortalamayı Geçti!
5 sayısının karesi 25 Ortalamayı Geçti!
6 sayısının karesi 36 Ortalamayı Geçti!
7 sayısının karesi 49 Ortalamayı Geçti!
8 sayısının karesi 64 Ortalamayı Geçti!
9 sayısının karesi 81 Ortalamayı Geçti!
*/

```

15.6 Düşün ve Çöz

- ♦ **Düşün:** LINQ'in 'ertelenmiş yürütme' (**deferred execution**) özelliği ne anlama gelir? Bir sorgu tanımlandığı anda değil, üzerinde foreach döngüsü kurulduğunda veya **.ToList()** çağrıldığında neden çalışır? Bu özelliğin hem avantajı hem de tuzağı nedir?
- ♦ **Düşün:** Sorgu söz dizimi (**query syntax**) ile akıcı söz dizimi (**method/fluent syntax**) arasındaki fark nedir? Hangi durumlarda biri diğerine üstün gelir? GroupJoin gibi karmaşık işlemler hangi söz diziminde daha okunabilir?
- ♦ **Düşün:** **IEnumerable<T>** üzerinde çalışan LINQ ile **IQueryable<T>** üzerinde çalışan LINQ arasındaki fark nedir? Veritabanı sorgulamada **IQueryable**'in 'sorgu sunucu tarafında derlenir' avantajı performansı nasıl etkiler?
- ♦ **Düşün:** **Select** ile **SelectMany** arasındaki fark nedir? İç içe koleksiyonları düzleştirme (**flatten**) gereken bir senaryoda **SelectMany** nasıl kullanılır?
- ♦ **Düşün:** LINQ'in **GroupBy** yöntemi hangi senaryolarda kullanılır? Gruplandırılmış veriler üzerinde ekleme (**aggregation**) yapmanın yolları nelerdir? SQL'deki **GROUP BY** ile karşılaştırınız.
- ♦ **Düşün:** LINQ sorguları döngülerle (**imperative**) yazılmış koda kıyasla performans açısından nasıldır? Her zaman LINQ kullanmak doğru mudur? LINQ'in okunabilirlik kazancını performans maliyetiyle dengelemenin kriterleri nelerdir?
- ♦ **Düşün:** **let** anahtar kelimesi LINQ içinde ara değişken tanımlamak için kullanılır. Bu özellik olmadan yazılması güçleşen sorgu türlerine örnek veriniz. **let**'in derleyici tarafından nasıl dönüştürüldüğünü düşününüz.
- ♦ **Çöz:** Bir öğrenci listesi üzerinde şu LINQ sorgularını yazınız: a) Ortalaması 70'in üzerinde olanlar, b) Sınıfa göre gruplanmış öğrenci sayıları, c) Her sınıftaki en yüksek not, d) Adı 'A' ile başlayanlar alfabetik sıralı.
- ♦ **Çöz:** İki liste arasında LINQ Join, GroupJoin ve Zip kullanarak veriyi birleştirin: Ürünler ve Kategoriler listesi verilsin; her ürünün kategori adını içeren birleşik liste oluşturun.
- ♦ **Çöz:** Akıcı söz dizimi kullanarak: bir metin içindeki sözcükleri uzunluklarına göre gruplandırın, her grubun en uzun sözcüğünü bulun ve sonuçları uzunluğa göre azalan sırayla listeleyin.
- ♦ **Çöz:** LINQ'in **Any**, **All**, **Count**, **Sum**, **Min**, **Max** ve **Average** yöntemlerinin her birini kullanan; bir sınav sonuçları listesi üzerinde istatistiksel bir rapor üreten program yazınız.
- ♦ **Çöz:** İç içe listeler içeren bir veri yapısı (örneğin her öğretmenin birden fazla dersi) üzerinde **SelectMany** kullanarak tüm dersleri düzleştiren ve LINQ ile filtreleyen bir sorgu yazınız.